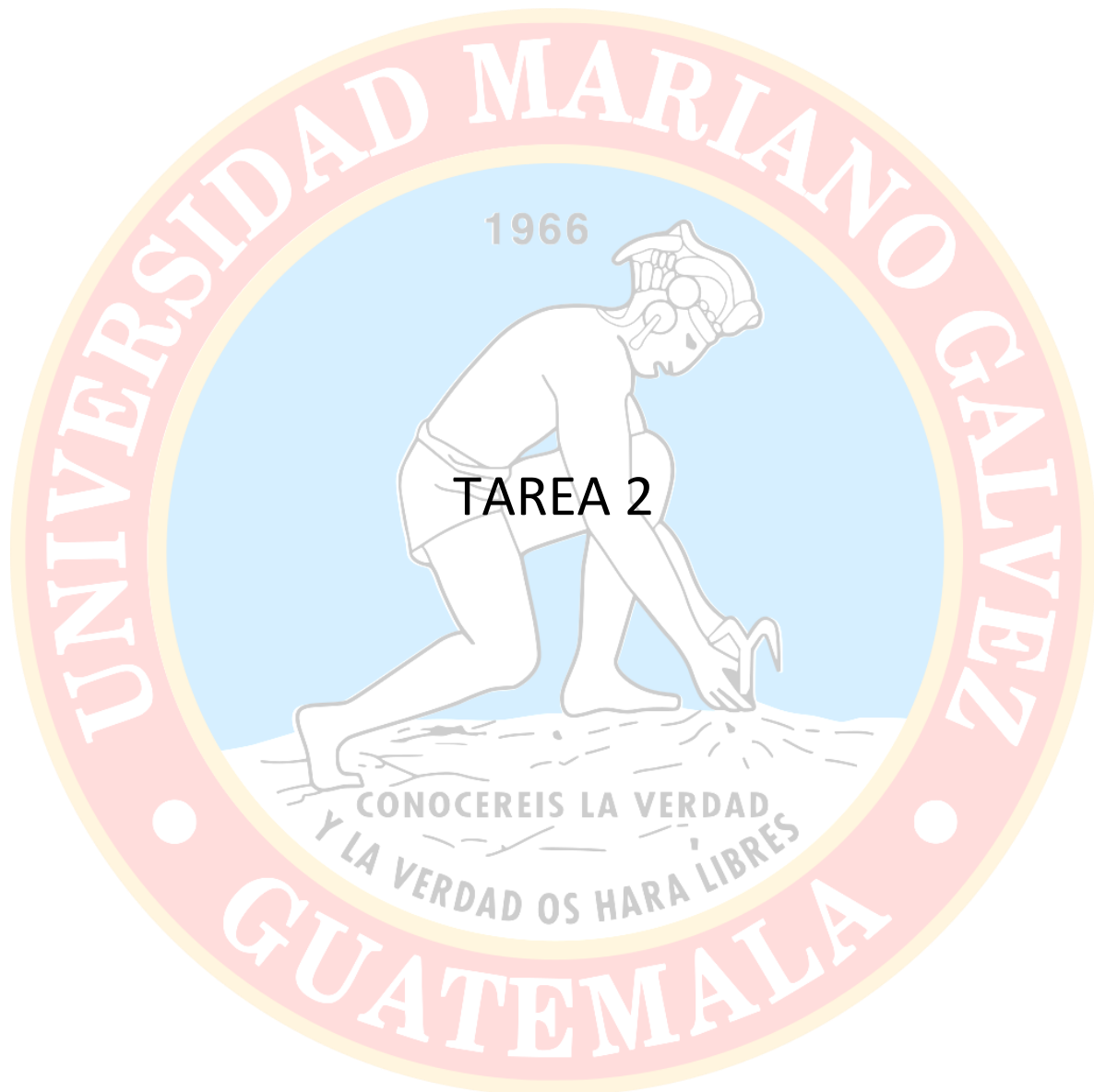


Universidad Mariano Galv  z, Boca del Monte, Guatemala Guatemala

Ingeniero: Melvin Cali

Curso: Programaci  n III

Actividad:



Angel Steve Morej  n Mart  nez

7690-23-21973

01/03/2026

INTRODUCCIÓN

En este programa vamos a identificar varios tipos de Ordenamientos: Selection sort (selección) Bubble sort (burbuja) Insertion sort (inserción) Merge sort (combinación) Quick sort (rápida) Heap sort (montón) Counting sort (conteo) Radix sort (raíz) Bucket sort (cubo), cada programa contiene un funcionamiento y diferentes niveles.

El programa a continuación permite al usuario ingresar una serie de números y seleccionar el método de ordenamiento que desea utilizar mediante un menú interactivo. Posteriormente, el sistema procesa los datos utilizando el algoritmo elegido y muestra el resultado ordenado en pantalla. De esta manera, el programa no solo permite observar el funcionamiento práctico de los algoritmos, sino que también facilita la comprensión de su comportamiento y rendimiento.

Estos son ampliamente utilizados en áreas como bases de datos, motores de búsqueda, análisis de información, sistemas administrativos y desarrollo de software en general. Comprender cómo funcionan estos métodos permite a los programadores seleccionar la técnica más adecuada para cada situación, optimizando así el desempeño de las aplicaciones y el manejo de grandes volúmenes de datos.



Analisis

Tipos de ordenamientos:

- **Selección Sort (Ordenamiento por selección)**
 - Su funcionamiento se basa en buscar repetidamente el elemento más pequeño dentro de una lista o arreglo y colocarlo en la posición correcta dentro de la secuencia ordenada. El proceso comienza seleccionando el primer elemento de la lista y comparándolo con todos los demás elementos para encontrar el valor mínimo. Una vez identificado, se intercambia con el elemento que se encuentra en la posición inicial.
- **Bubble Sort (Ordenamiento de burbuja)**
 - Es uno de los algoritmos de ordenamiento más conocidos debido a su simplicidad. Su nombre proviene del comportamiento que presentan los elementos durante el proceso de ordenamiento, ya que los valores más grandes tienden a “subir” hacia el final de la lista, de manera similar a como las burbujas ascienden en el agua.

El algoritmo funciona comparando pares de elementos adyacentes dentro de la lista. Si el elemento actual es mayor que el siguiente, se realiza un intercambio entre ellos. Este proceso se repite continuamente recorriendo toda la lista hasta que no se necesiten más intercambios, lo que indica que el arreglo ya está ordenado.
- **Insertion Sort (Ordenamiento por inserción)**
 - Funciona de manera similar a la forma en que una persona ordena cartas en su mano. El proceso consiste en tomar un elemento del arreglo y colocarlo en la posición correcta dentro de una sección que ya se encuentra ordenada.

Comienza considerando el segundo elemento del arreglo, ya que el primero se asume inicialmente como ordenado. Luego compara ese elemento con el anterior y lo inserta en la posición correspondiente dentro de la subsecuencia ordenada. Este procedimiento se repite para cada elemento del arreglo hasta que todos han sido ubicados correctamente.
- **Merge Sort (Ordenamiento por Mezcla)**
 - Basado en la técnica conocida como **Divide y Vencerás**. Su funcionamiento consiste en dividir repetidamente el arreglo original en partes más pequeñas hasta que cada sublista contenga únicamente un elemento. Posteriormente, estas sublistas se combinan nuevamente de manera ordenada hasta reconstruir el arreglo completo. Durante el proceso de combinación, el algoritmo compara los elementos de las sublistas y los va colocando en el orden correcto dentro de una nueva estructura temporal. Este proceso garantiza que cada paso de combinación genere una lista ordenada.

- Quick Sort (Ordenamiento Rápido)

- Es uno de los algoritmos de ordenamiento más eficientes y utilizados en la práctica. Al igual que el Merge Sort, también utiliza la estrategia de **Divide y Vencerás**, pero su método de división es diferente.

El algoritmo selecciona un elemento del arreglo llamado **pivote**. Luego reorganiza los demás elementos de manera que todos los valores menores al pivote se ubiquen a su izquierda, mientras que los valores mayores se coloquen a su derecha. Una vez realizada esta partición, el algoritmo aplica el mismo procedimiento de forma recursiva a cada una de las sublistas generadas.

- Heap Sort (Ordenamiento por Montículo)

- Se basa en una estructura de datos conocida como **montículo o heap**, que generalmente se representa mediante un árbol binario completo. En este tipo de estructura, el valor del nodo padre siempre es mayor o menor que el de sus nodos hijos, dependiendo del tipo de heap utilizado.

El algoritmo comienza transformando el arreglo en un **heap máximo**, donde el valor más grande se encuentra en la raíz del árbol. Luego, el elemento mayor se intercambia con el último elemento del arreglo, reduciendo el tamaño del heap. Este proceso se repite hasta que todos los elementos han sido colocados en su posición correcta.

- Counting Sort (Ordenamiento por Conteo)

- Funciona de manera diferente a los métodos tradicionales basados en comparaciones. En lugar de comparar elementos entre sí, este algoritmo cuenta la cantidad de veces que aparece cada valor dentro del arreglo.

Primero se crea un arreglo auxiliar que almacena el número de ocurrencias de cada elemento. Luego, utilizando esta información, se reconstruye el arreglo original colocando cada valor en la posición correspondiente según la cantidad de veces que aparece.

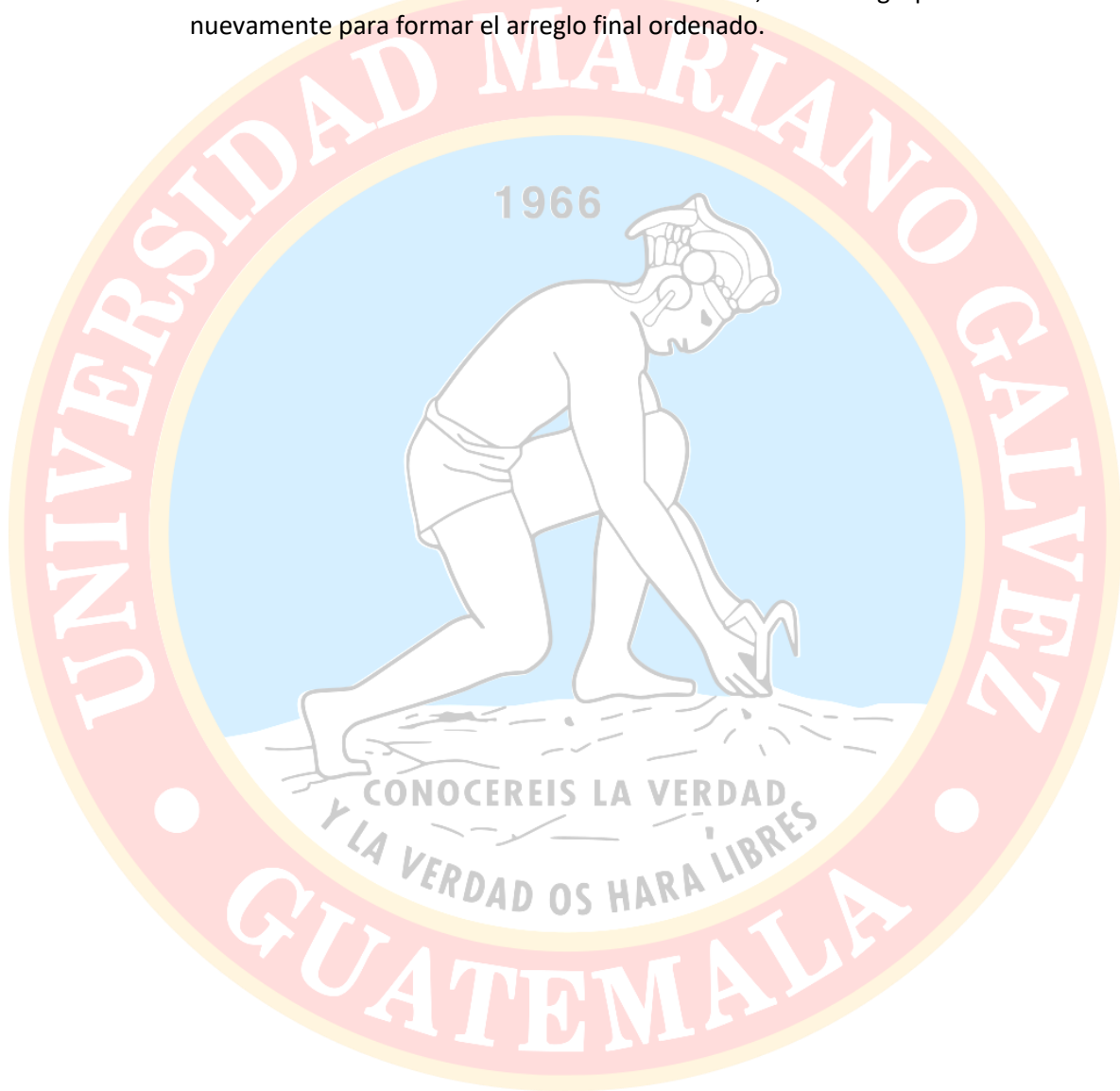
- Radix Sort (Ordenamiento por Raíz)

- El **Radix Sort** es un algoritmo de ordenamiento que también evita el uso de comparaciones directas entre elementos. En lugar de ello, organiza los números procesando cada uno de sus dígitos individualmente, comenzando generalmente desde el dígito menos significativo hasta el más significativo.

El algoritmo agrupa los números según el valor de cada dígito utilizando un método auxiliar, comúnmente Counting Sort. Este proceso se repite para cada posición decimal del número hasta que todos los dígitos han sido evaluados.

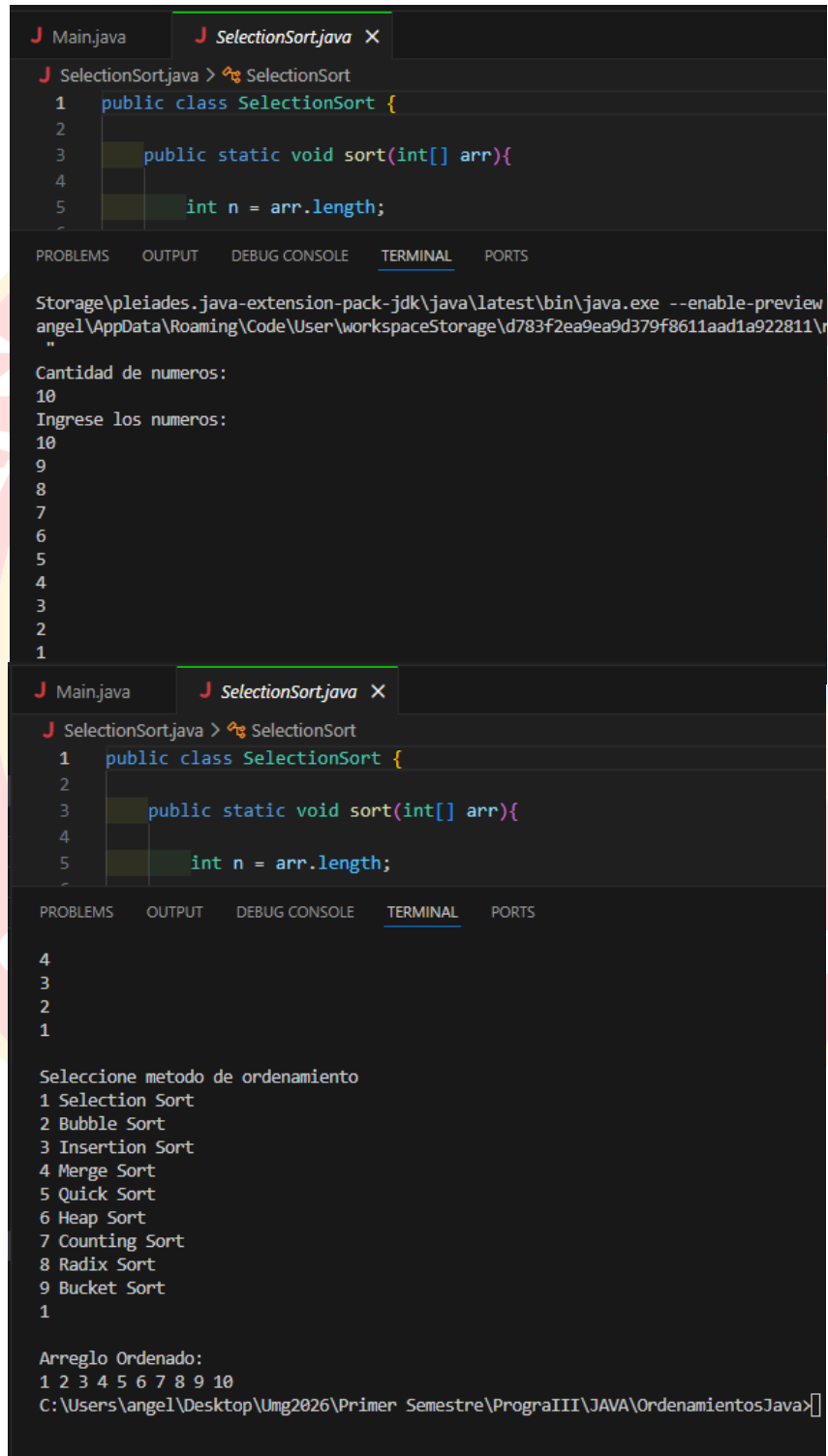
- Bucket Sort (Ordenamiento por Cubetas)
 - El **Bucket Sort** es un algoritmo que divide los elementos del arreglo en varios grupos llamados **cubetas o buckets**. Cada cubeta almacena un subconjunto de los datos, generalmente basándose en un rango específico de valores.

Una vez que los elementos han sido distribuidos en las diferentes cubetas, cada grupo se ordena individualmente utilizando otro algoritmo de ordenamiento, como Insertion Sort o Collections Sort. Finalmente, todos los grupos se combinan nuevamente para formar el arreglo final ordenado.



FUNCIONAMIENTOS DE CADA ORDEN

- 1. Selection Sort (Ordenamiento por Selección)



```
J Main.java J SelectionSort.java X
J SelectionSort.java > SelectionSort
1 public class SelectionSort {
2
3     public static void sort(int[] arr){
4
5         int n = arr.length;

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Storage\pleiades.java-extension-pack-jdk\java\latest\bin\java.exe --enable-preview
angel\AppData\Roaming\Code\User\workspaceStorage\d783f2ea9ea9d379f8611aad1a922811\
"

Cantidad de numeros:
10
Ingrese los numeros:
10
9
8
7
6
5
4
3
2
1

J Main.java J SelectionSort.java X
J SelectionSort.java > SelectionSort
1 public class SelectionSort {
2
3     public static void sort(int[] arr){
4
5         int n = arr.length;

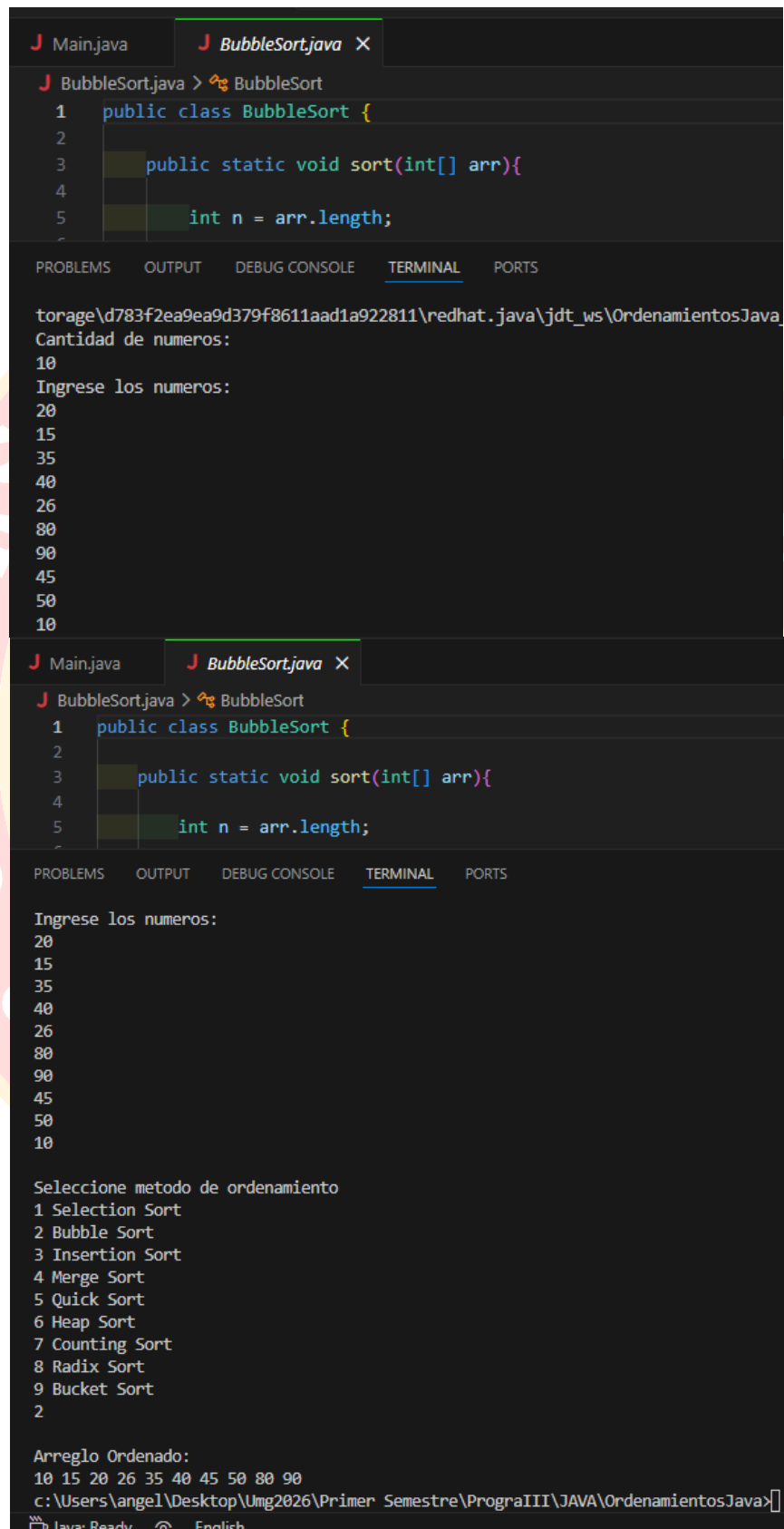
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

4
3
2
1

Seleccione metodo de ordenamiento
1 Selection Sort
2 Bubble Sort
3 Insertion Sort
4 Merge Sort
5 Quick Sort
6 Heap Sort
7 Counting Sort
8 Radix Sort
9 Bucket Sort
1

Arreglo Ordenado:
1 2 3 4 5 6 7 8 9 10
C:\Users\angel\Desktop\Umg2026\Primer Semestre\PrograIII\JAVA\OrdenamientosJava>
```

- 2. Bubble Sort (Ordenamiento de Burbuja)



```
J Main.java J BubbleSort.java X
J BubbleSort.java > BubbleSort
1 public class BubbleSort {
2
3     public static void sort(int[] arr){
4
5         int n = arr.length;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

torage\d783f2ea9ea9d379f8611aad1a922811\redhat.java\jdt_ws\OrdenamientosJava

Cantidad de numeros:
10
Ingrese los numeros:
20
15
35
40
26
80
90
45
50
10

```
J Main.java J BubbleSort.java X
J BubbleSort.java > BubbleSort
1 public class BubbleSort {
2
3     public static void sort(int[] arr){
4
5         int n = arr.length;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

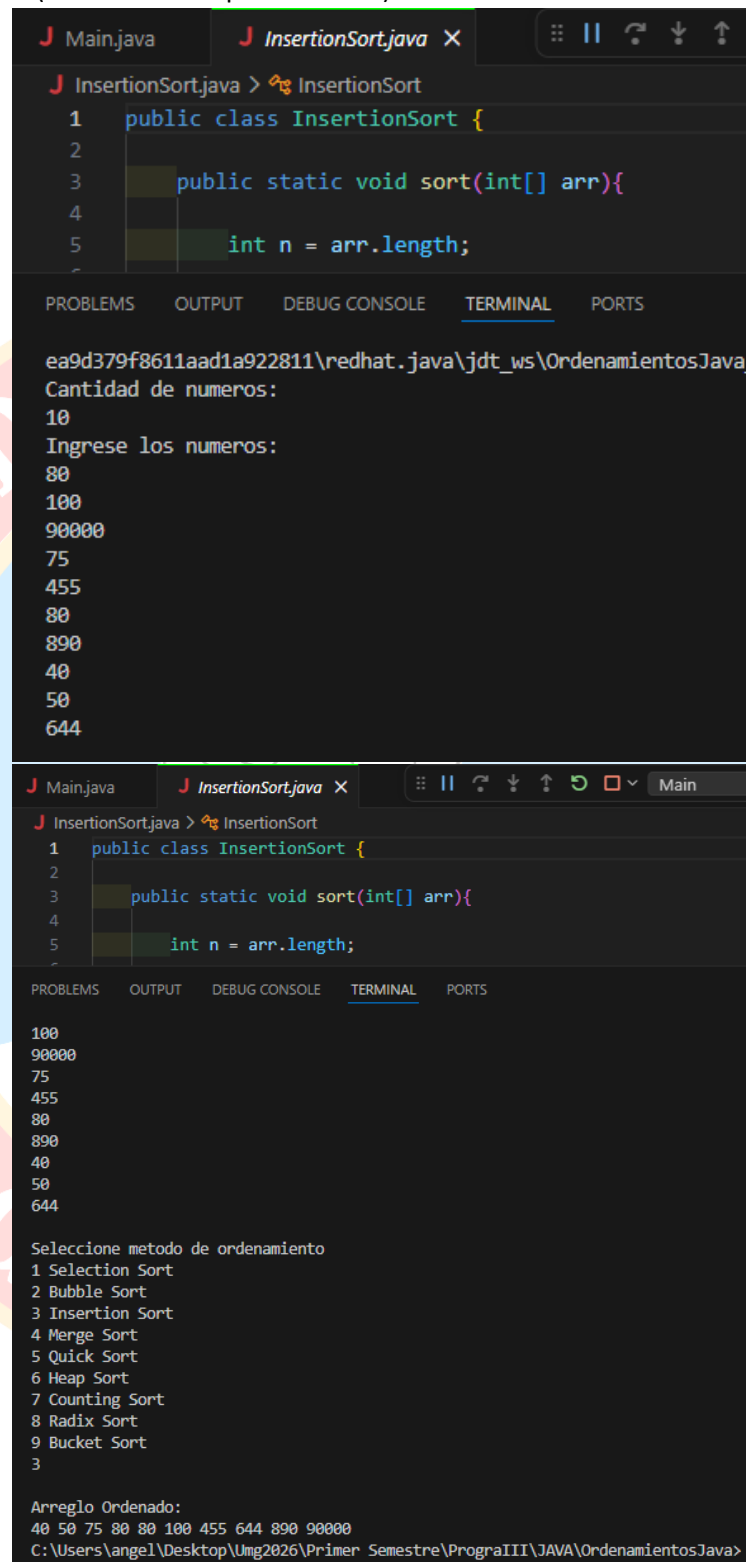
Ingrese los numeros:
20
15
35
40
26
80
90
45
50
10

Seleccione metodo de ordenamiento
1 Selection Sort
2 Bubble Sort
3 Insertion Sort
4 Merge Sort
5 Quick Sort
6 Heap Sort
7 Counting Sort
8 Radix Sort
9 Bucket Sort
2

Arreglo Ordenado:
10 15 20 26 35 40 45 50 80 90
c:\Users\angel\Desktop\Umg2026\Primer Semestre\ProgrnaIII\JAVA\OrdenamientosJava>

Java: Ready English

- 3. Insertion Sort (Ordenamiento por Inserción)



```
J Main.java J InsertionSort.java X
J InsertionSort.java > InsertionSort
1 public class InsertionSort {
2
3     public static void sort(int[] arr){
4
5         int n = arr.length;
```

ea9d379f8611aad1a922811\redhat.java\jdt_ws\OrdenamientosJava

Cantidad de numeros:
10
Ingrese los numeros:
80
100
90000
75
455
80
890
40
50
644

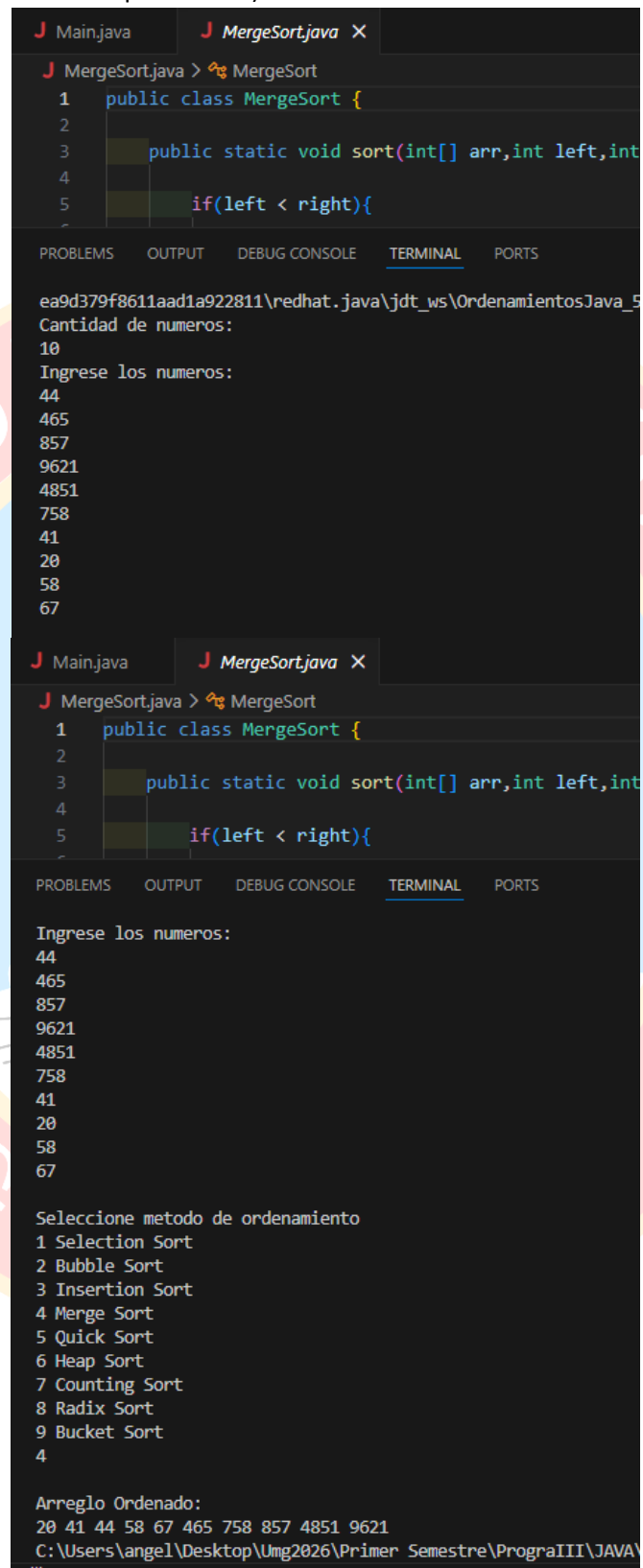
```
J Main.java J InsertionSort.java X Main
J InsertionSort.java > InsertionSort
1 public class InsertionSort {
2
3     public static void sort(int[] arr){
4
5         int n = arr.length;
```

100
90000
75
455
80
890
40
50
644

Seleccione metodo de ordenamiento
1 Selection Sort
2 Bubble Sort
3 Insertion Sort
4 Merge Sort
5 Quick Sort
6 Heap Sort
7 Counting Sort
8 Radix Sort
9 Bucket Sort
3

Arreglo Ordenado:
40 50 75 80 80 100 455 644 890 90000
C:\Users\angel\Desktop\Umg2026\Primer Semestre\PrograIII\JAVA\OrdenamientosJava>

- Merge Sort (Ordenamiento por Mezcla)



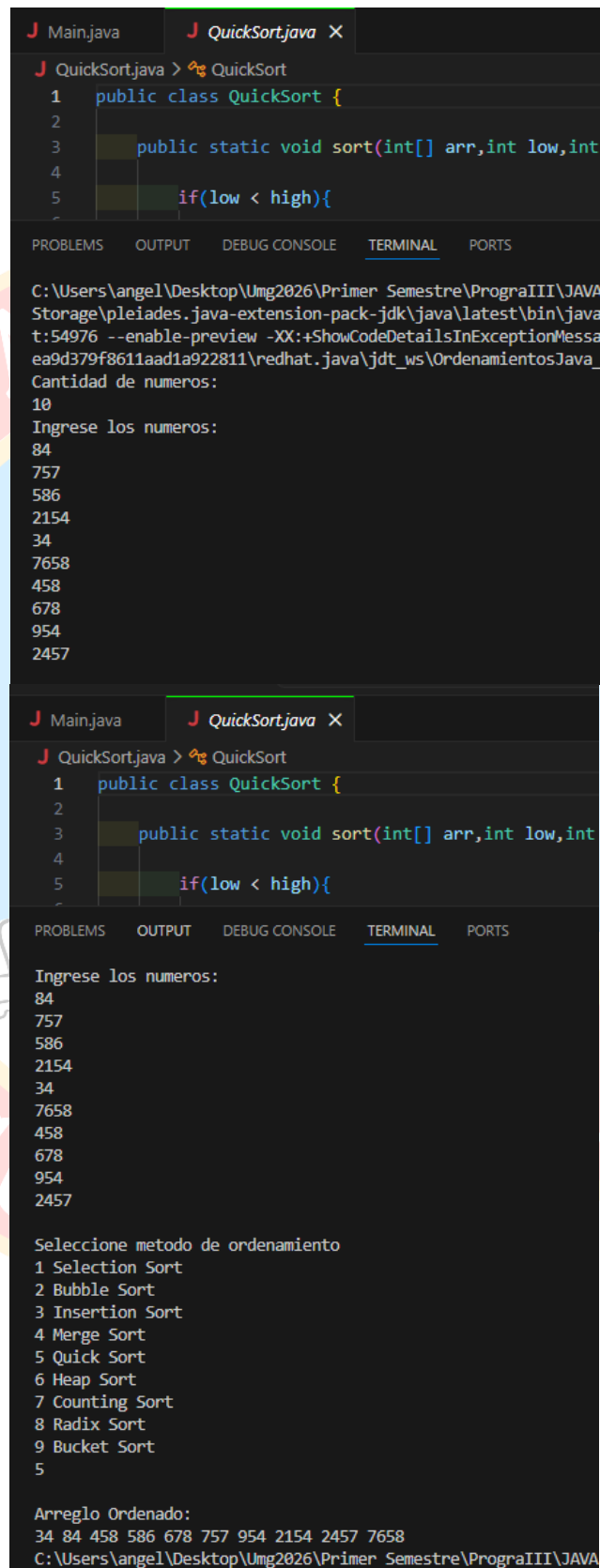
The screenshot shows an IDE with two tabs: `Main.java` and `MergeSort.java`. The `MergeSort.java` tab is active, displaying the following code:

```
1 public class MergeSort {  
2  
3     public static void sort(int[] arr,int left,int  
4  
5     if(left < right){
```

Below the code editor, the `TERMINAL` tab is selected, showing the execution of the program. The output is as follows:

```
ea9d379f8611aad1a922811\redhat.java\jdt_ws\OrdenamientosJava_5  
Cantidad de numeros:  
10  
Ingrese los numeros:  
44  
465  
857  
9621  
4851  
758  
41  
20  
58  
67  
  
Ingrese los numeros:  
44  
465  
857  
9621  
4851  
758  
41  
20  
58  
67  
  
Seleccione metodo de ordenamiento  
1 Selection Sort  
2 Bubble Sort  
3 Insertion Sort  
4 Merge Sort  
5 Quick Sort  
6 Heap Sort  
7 Counting Sort  
8 Radix Sort  
9 Bucket Sort  
4  
  
Arreglo Ordenado:  
20 41 44 58 67 465 758 857 4851 9621  
C:\Users\angel\Desktop\Umg2026\Primer Semestre\PrograIII\JAVA\
```

- Quick Sort (Ordenamiento Rápido)



```
J Main.java J QuickSort.java X
J QuickSort.java > QuickSort
1 public class QuickSort {
2
3     public static void sort(int[] arr,int low,int
4
5     if(low < high){
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

C:\Users\angel\Desktop\Umg2026\Primer Semestre\PrograIII\JAVA
Storage\pleiades.java-extension-pack-jdk\java\latest\bin\java
t:54976 --enable-preview -XX:+ShowCodeDetailsInExceptionMessa
ea9d379f8611aad1a922811\redhat.java\jdt_ws\OrdenamientosJava_
Cantidad de numeros:
10
Ingrese los numeros:
84
757
586
2154
34
7658
458
678
954
2457

J Main.java J QuickSort.java X
J QuickSort.java > QuickSort
1 public class QuickSort {
2
3     public static void sort(int[] arr,int low,int
4
5     if(low < high){
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

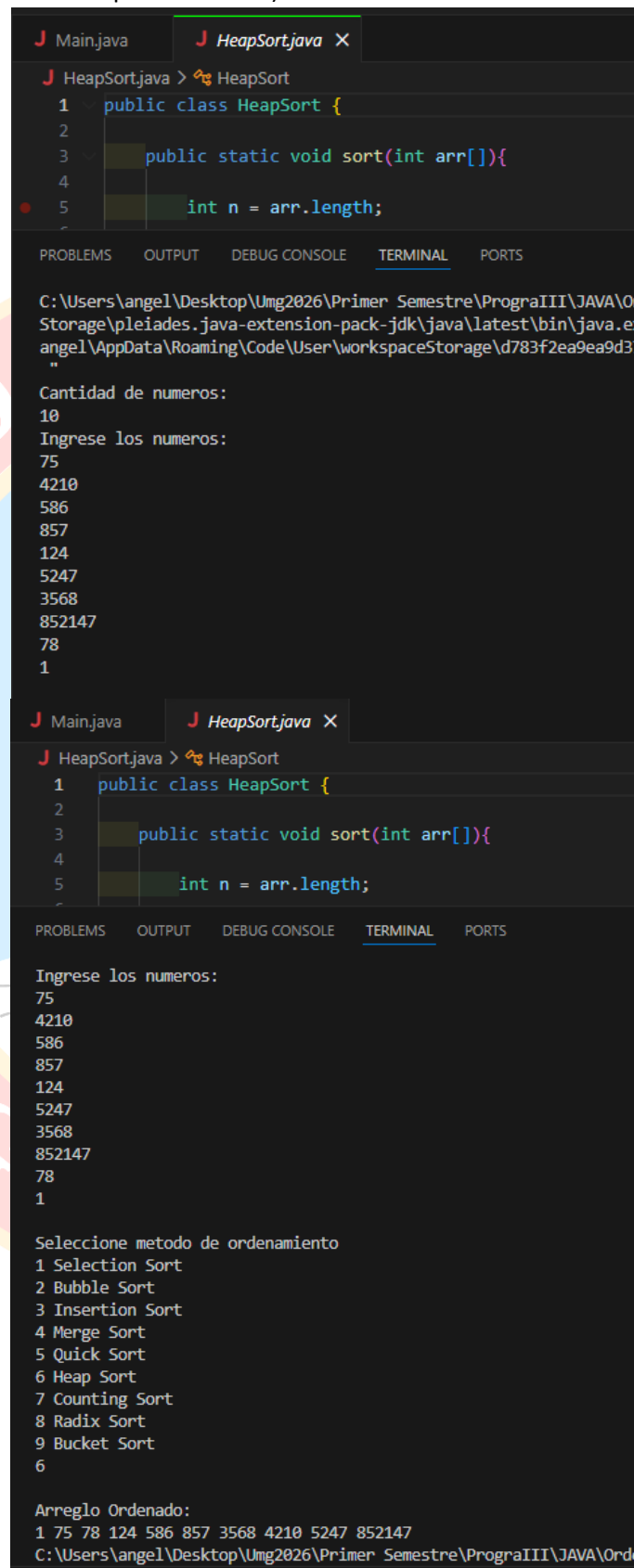
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Ingrese los numeros:
84
757
586
2154
34
7658
458
678
954
2457

Seleccione metodo de ordenamiento
1 Selection Sort
2 Bubble Sort
3 Insertion Sort
4 Merge Sort
5 Quick Sort
6 Heap Sort
7 Counting Sort
8 Radix Sort
9 Bucket Sort
5

Arreglo Ordenado:
34 84 458 586 678 757 954 2154 2457 7658
C:\Users\angel\Desktop\Umg2026\Primer Semestre\PrograIII\JAVA
```

- Heap Sort (Ordenamiento por Montículo)



```
J Main.java J HeapSort.java X
J HeapSort.java > HeapSort
1 public class HeapSort {
2
3     public static void sort(int arr[]){
4
5         int n = arr.length;
6
7     }
8 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

C:\Users\angel\Desktop\Umg2026\Primer Semestre\PrograIII\JAVA\Ord
Storage\pleiades.java-extension-pack-jdk\java\latest\bin\java.e
angel\AppData\Roaming\Code\User\workspaceStorage\d783f2ea9ea9d3
"

Cantidad de numeros:
10
Ingrese los numeros:
75
4210
586
857
124
5247
3568
852147
78
1

J Main.java J HeapSort.java X
J HeapSort.java > HeapSort
1 public class HeapSort {
2
3     public static void sort(int arr[]){
4
5         int n = arr.length;
6
7     }
8 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Ingrese los numeros:
75
4210
586
857
124
5247
3568
852147
78
1

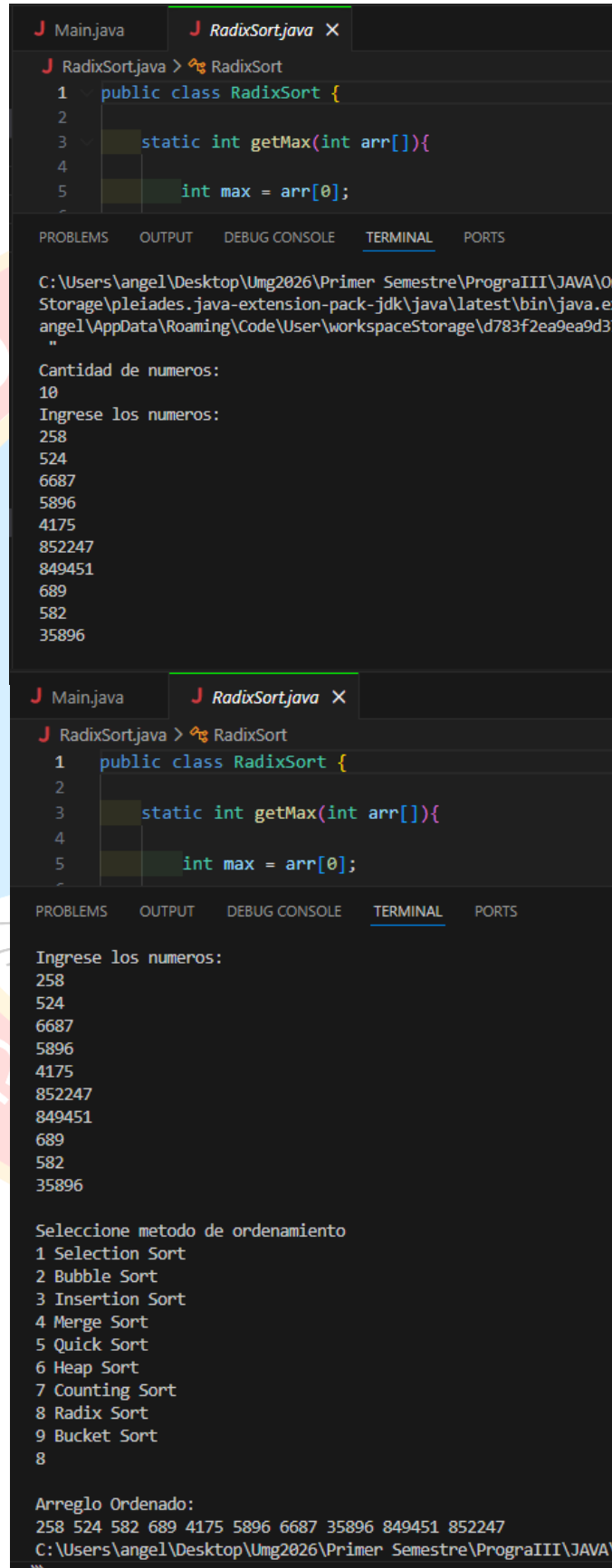
Seleccione metodo de ordenamiento
1 Selection Sort
2 Bubble Sort
3 Insertion Sort
4 Merge Sort
5 Quick Sort
6 Heap Sort
7 Counting Sort
8 Radix Sort
9 Bucket Sort
6

Arreglo Ordenado:
1 75 78 124 586 857 3568 4210 5247 852147
C:\Users\angel\Desktop\Umg2026\Primer Semestre\PrograIII\JAVA\Ord
```

- ```
C:\Users\angel\Desktop\img2026\Primer_Semestre\Programas\Java\workspace\pleiades.java-extension-pack-jdk\java\latest\bin\java.
angel\AppData\Roaming\Code\User\workspaceStorage\d783f2ea9ea9d
"
Cantidad de numeros:
10
Ingrese los numeros:
465
58
161
465
48262
5896
357
852
951
758469
J Main.java J CountingSort.java X
J CountingSort.java > CountingSort
1 public class CountingSort {
2
3 public static void sort(int[] arr){
4
5 int max = arr[0];
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
7
```

```
CountingSort.java > CountingSort
1 public class CountingSort {
2
3 public static void sort(int[] arr){
4
5 int max = arr[0];
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
```

- Radix Sort (Ordenamiento por Raíz)



```
J Main.java J RadixSort.java X
J RadixSort.java > RadixSort
1 public class RadixSort {
2
3 static int getMax(int arr[]){
4
5 int max = arr[0];
6
7 }
8 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

C:\Users\angel\Desktop\Umg2026\Primer Semestre\PrograIII\JAVA\O
Storage\pleiades.java-extension-pack-jdk\java\latest\bin\java.e
angel\AppData\Roaming\Code\User\workspaceStorage\d783f2ea9ea9d3
"
Cantidad de numeros:
10
Ingrese los numeros:
258
524
6687
5896
4175
852247
849451
689
582
35896

J Main.java J RadixSort.java X
J RadixSort.java > RadixSort
1 public class RadixSort {
2
3 static int getMax(int arr[]){
4
5 int max = arr[0];
6
7 }
8 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

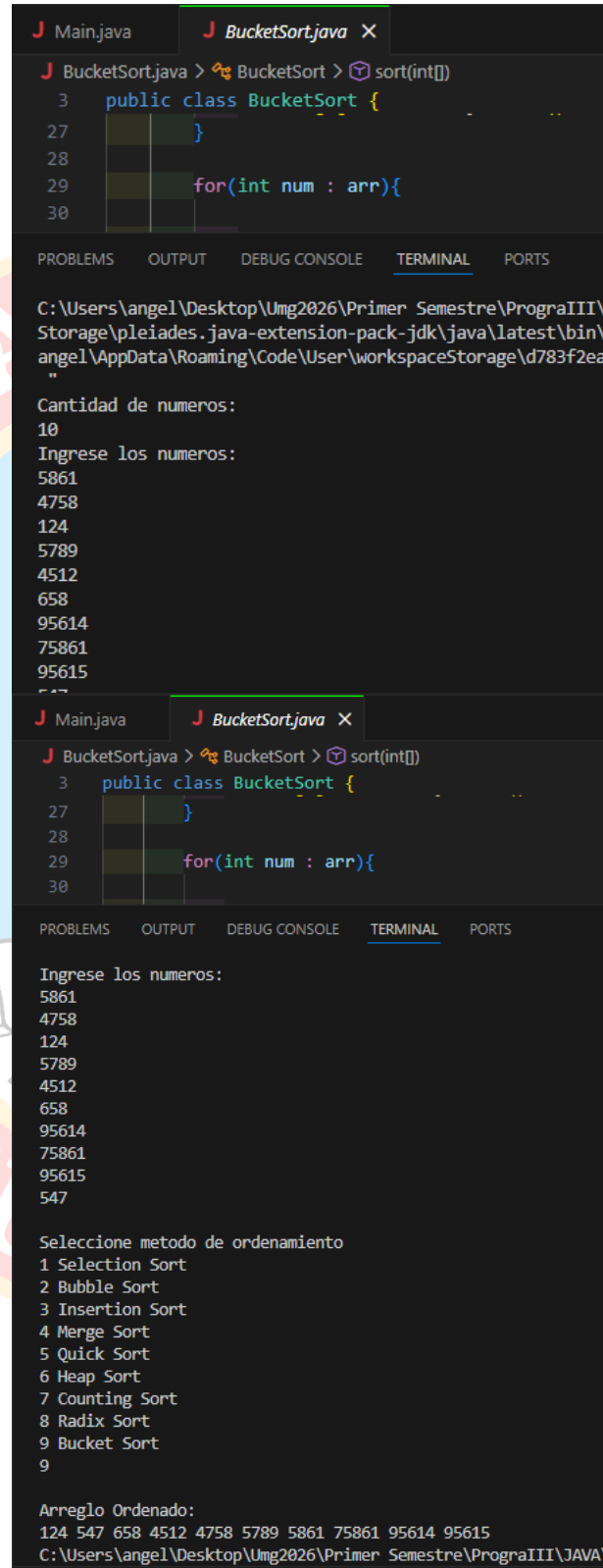
Ingrese los numeros:
258
524
6687
5896
4175
852247
849451
689
582
35896

Seleccione metodo de ordenamiento
1 Selection Sort
2 Bubble Sort
3 Insertion Sort
4 Merge Sort
5 Quick Sort
6 Heap Sort
7 Counting Sort
8 Radix Sort
9 Bucket Sort
8

Arreglo Ordenado:
258 524 582 689 4175 5896 6687 35896 849451 852247
C:\Users\angel\Desktop\Umg2026\Primer Semestre\PrograIII\JAVA\
...

```

- Bucket Sort (Ordenamiento por Cubetas)



```

J Main.java J BucketSort.java X
J BucketSort.java > BucketSort > sort(int[])
3 public class BucketSort {
27 }
28
29 for(int num : arr){
30
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
C:\Users\angel\Desktop\Umg2026\Primer Semestre\PrograIII\
Storage\pleiades.java-extension-pack-jdk\java\latest\bin\
angel\AppData\Roaming\Code\User\workspaceStorage\d783f2ea
"
Cantidad de numeros:
10
Ingrese los numeros:
5861
4758
124
5789
4512
658
95614
75861
95615
547

J Main.java J BucketSort.java X
J BucketSort.java > BucketSort > sort(int[])
3 public class BucketSort {
27 }
28
29 for(int num : arr){
30

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Ingrese los numeros:
5861
4758
124
5789
4512
658
95614
75861
95615
547

Seleccione metodo de ordenamiento
1 Selection Sort
2 Bubble Sort
3 Insertion Sort
4 Merge Sort
5 Quick Sort
6 Heap Sort
7 Counting Sort
8 Radix Sort
9 Bucket Sort
9

Arreglo Ordenado:
124 547 658 4512 4758 5789 5861 75861 95614 95615
C:\Users\angel\Desktop\Umg2026\Primer Semestre\PrograIII\JAVA

```

## Conclusión

La elaboración del programa permitió observar cómo diferentes algoritmos pueden resolver el mismo problema, pero utilizando estrategias distintas. Algunos métodos, como Bubble Sort o Selection Sort, resultan más sencillos de comprender y programar, aunque su eficiencia es menor cuando se trabaja con grandes cantidades de datos. Por otro lado, algoritmos más avanzados como Merge Sort, Quick Sort o Heap Sort ofrecen un mejor rendimiento y son ampliamente utilizados en aplicaciones reales donde se requiere procesar grandes volúmenes de información.

Asimismo, este proyecto permitió reforzar conocimientos relacionados con la programación en Java, el uso de estructuras de datos como arreglos, la creación de clases y métodos, y la implementación de menús interactivos para el usuario. También se pudo apreciar la importancia de elegir el algoritmo adecuado dependiendo del tipo de datos y del contexto en el que se utilice el programa.

